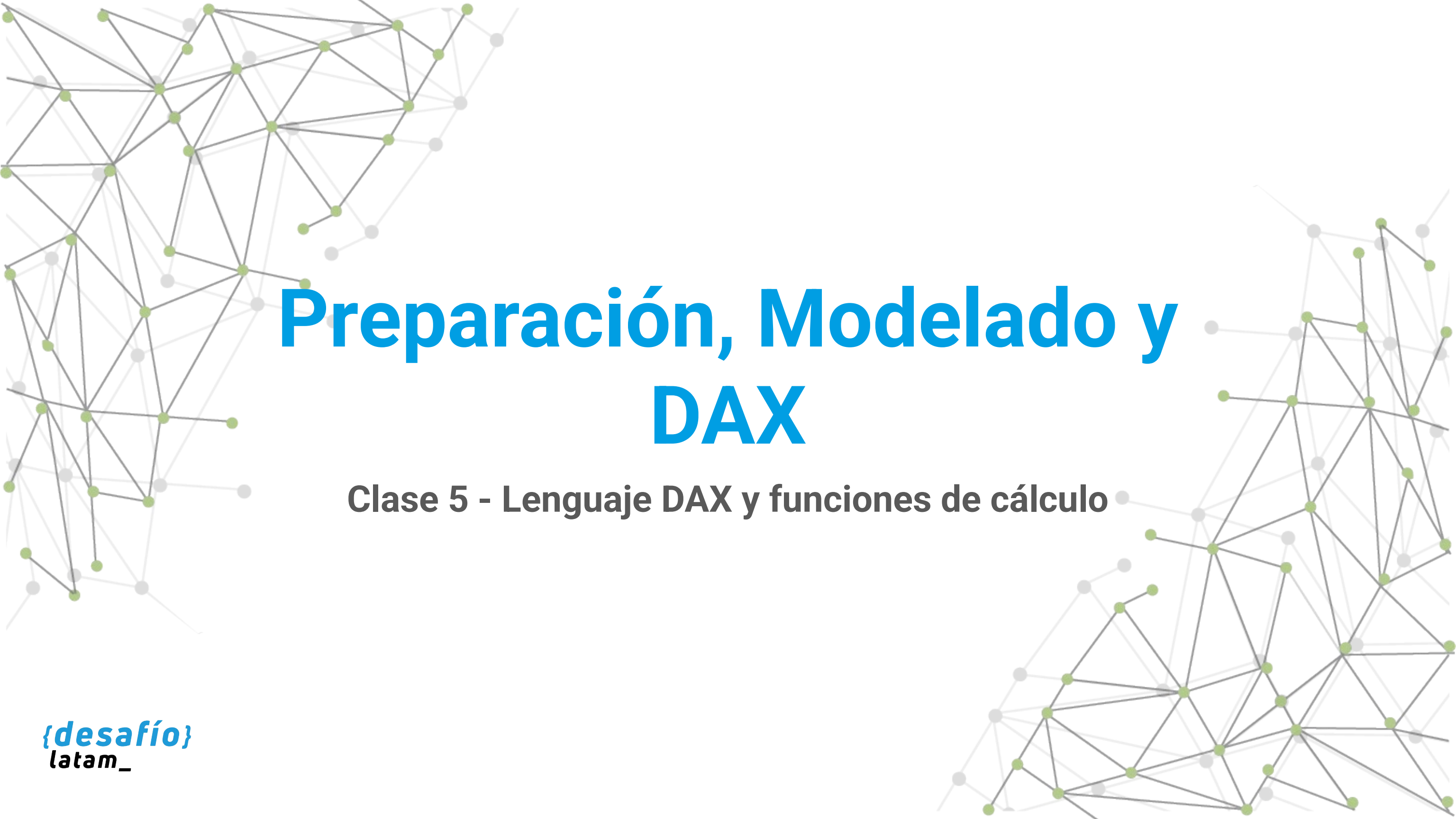




Preparación, Modelado y DAX

Clase 5 - Lenguaje DAX y funciones de cálculo



¿Qué aprenderás en esta sesión?

Aprenderemos a utilizar el lenguaje DAX (Data Analysis Expressions) en Power BI para la creación de columnas, medidas y tablas calculadas. Exploraremos funciones esenciales para el análisis de datos, incluyendo funciones de agregación, relacionales, condicionales, iterativas, de inteligencia de tiempo, de texto y estadísticas.

Desafío inicial

Has construido un modelo de datos funcional con relaciones claras entre hechos y dimensiones. Sin embargo, te enfrentas al reto de generar indicadores clave que no están disponibles directamente en las tablas. Por ejemplo:



Calcular el promedio de resolución de solicitudes para una comuna específica.



Comparar el número de solicitudes actuales con el mismo periodo del año anterior.



Crear una tabla de clasificación según tiempos de atención.

Para responder a estos requerimientos, necesitarás dominar el lenguaje DAX y sus funciones.

Esta sesión te proporcionará las herramientas necesarias para transformar tus modelos en sistemas analíticos verdaderamente dinámicos y flexibles.

/* DAX */

¿Qué es DAX y por qué utilizarlo?

¿De qué manera un lenguaje especializado como DAX puede potenciar tus modelos de datos más allá de lo que permite una base de datos relacional?

DAX (Data Analysis Expressions) es un lenguaje de fórmulas y expresiones diseñado específicamente para trabajar con datos tabulares en herramientas como Power BI, Power Pivot y Analysis Services. Su propósito es permitir el cálculo dinámico de valores agregados, indicadores personalizados y métricas complejas que dependen del contexto de los datos en uso.

DAX permite crear:

1

Columnas calculadas

Se evalúan fila por fila y expanden las tablas existentes.

2

Medidas

Cálculos agregados que responden dinámicamente al filtro aplicado.

3

Tablas calculadas

Estructuras derivadas que se comportan como cualquier tabla en el modelo.

En entornos profesionales, como servicios municipales o instituciones públicas, los datos suelen almacenarse en registros simples. Sin embargo, los indicadores exigidos por las jefaturas, auditorías o stakeholders requieren comparaciones temporales, tasas porcentuales, métricas acumuladas, entre otras. Sin DAX, estas transformaciones tendrían que realizarse fuera de Power BI, rompiendo el principio de autoservicio analítico.

Diferencias clave con fórmulas de Excel

Característica	Excel	DAX
Contexto de cálculo	Celdas específicas	Filtro aplicado en el modelo
Rendimiento	Local, no optimizado	Motor de cálculo columnar optimizado
Alcance	Solo archivos individuales	Modelos tabulares con relaciones
Reutilización	Fórmulas locales	Medidas reutilizables y dinámicas

Tabla 1. Comparación entre fórmulas en Excel y expresiones DAX

Fuente: Desafío Latam

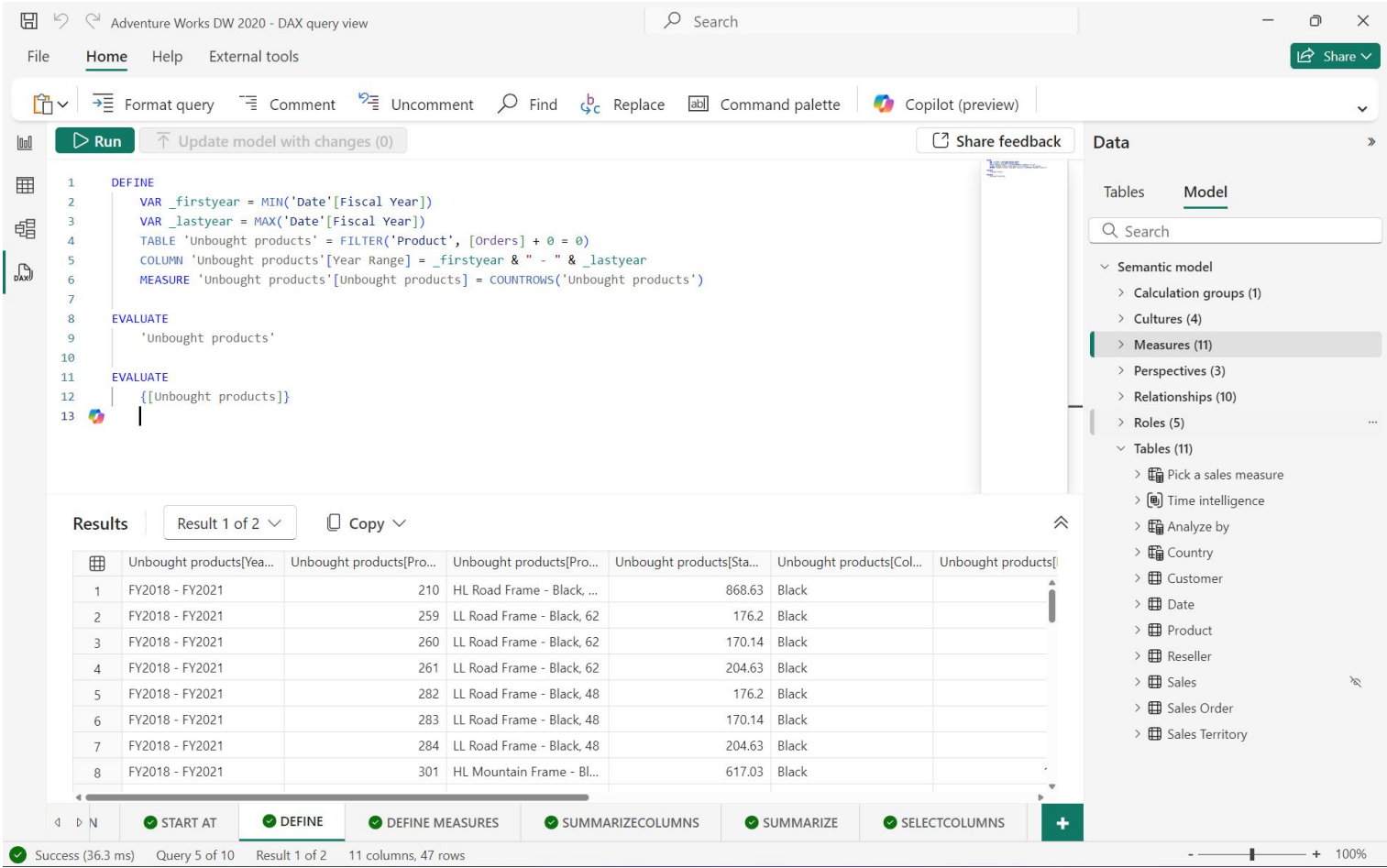
Ejemplo inicial

Una medida básica en DAX puede lucir así:

```
Total_Solicitudes =  
COUNTROWS(Solicitudes)
```

Y su resultado cambiará automáticamente según el filtro aplicado (por comuna, mes, servicio, etc.), lo que permite construir paneles verdaderamente interactivos.

DAX no es solo un lenguaje técnico; es una herramienta clave para transformar datos en conocimiento accionable dentro de Power BI. Su integración con modelos relacionales permite generar indicadores sólidos y contextualizados. Con esta base, avanzaremos hacia la creación concreta de columnas, medidas y tablas utilizando DAX.

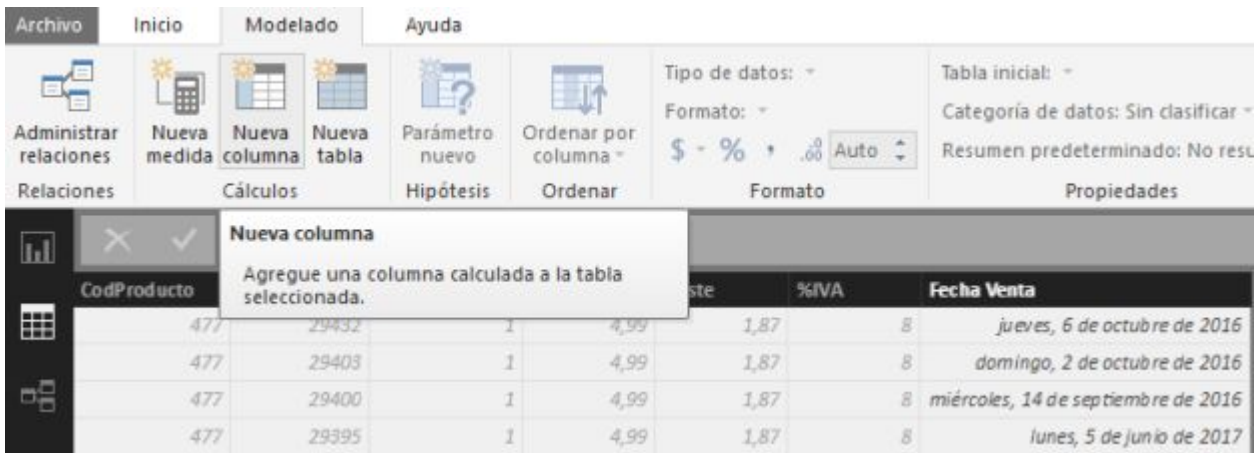


Creación de columnas calculadas

¿Cómo podemos enriquecer nuestras tablas con información nueva que no está explícitamente presente en los datos originales?

Las columnas calculadas son campos adicionales que se crean dentro de una tabla del modelo y se calculan fila por fila utilizando expresiones DAX. A diferencia de las medidas, que operan sobre agregados, las columnas calculadas amplían directamente la estructura de los datos, permitiendo nuevas categorizaciones o valores derivados.

Se almacenan físicamente en el modelo y, por tanto, deben usarse con criterio para no afectar el rendimiento.



En una institución pública, una columna calculada puede ser útil para clasificar solicitudes ciudadanas según el tiempo de respuesta, permitiendo segmentaciones posteriores en visualizaciones y análisis.

Buenas prácticas

Evita usar columnas calculadas cuando el mismo resultado puede lograrse con una medida.

Utiliza nombres descriptivos y coherentes con el dominio del negocio.

Comprueba que la lógica funcione correctamente en todas las filas (validación de bordes).

No dupliques la lógica: reutiliza expresiones mediante variables (VAR) si es necesario.

Errores comunes

- Usar columnas calculadas innecesariamente cuando se necesita un cálculo agregado.
- Aplicar expresiones que dependen del contexto de filtro, lo que puede generar valores incorrectos por fila.
- No considerar los tipos de datos (por ejemplo, comparar fechas con texto).

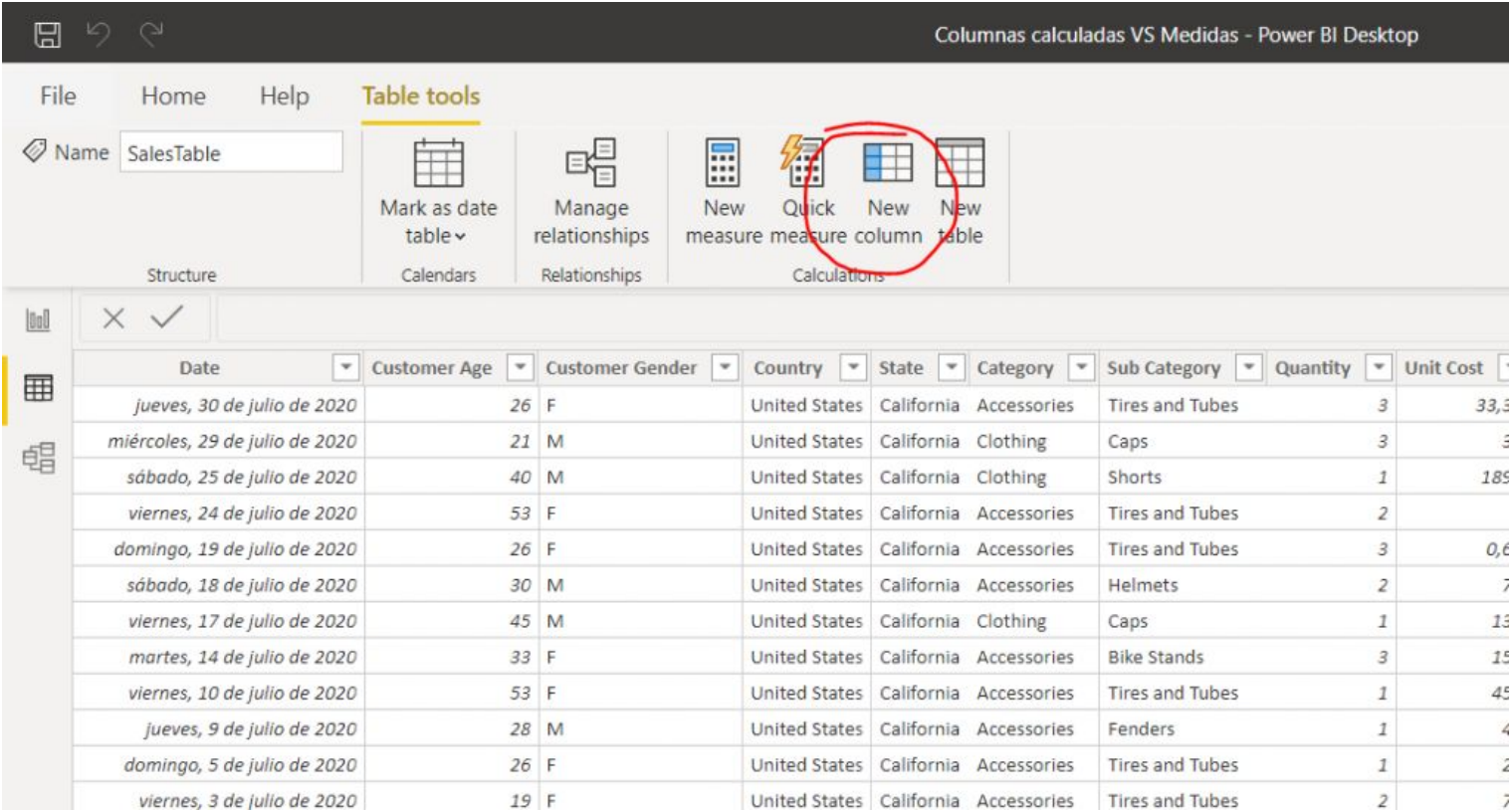
/* Medidas calculadas */

Creación de medidas calculadas

¿Cómo podemos obtener indicadores agregados que cambien dinámicamente según los filtros aplicados en un informe?

Las medidas son expresiones DAX que realizan cálculos agregados sobre las tablas del modelo, como sumas, promedios, máximos, conteos o cálculos personalizados. A diferencia de las columnas calculadas, las medidas no se almacenan fila a fila: se evalúan en tiempo real, según el contexto del filtro en las visualizaciones.

Las medidas son esenciales para construir dashboards interactivos y eficientes en Power BI.



Ejemplo aplicado

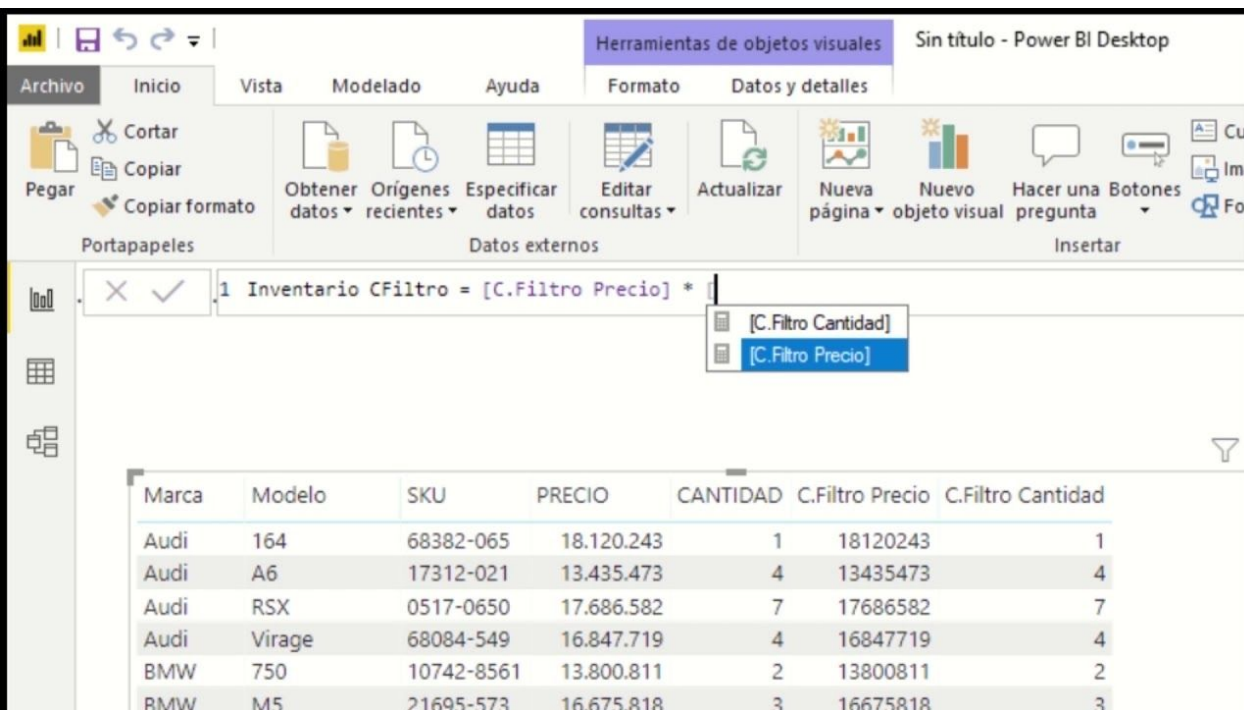
Supongamos que queremos calcular la cantidad total de solicitudes registradas en una tabla Solicitudes. Podríamos crear la siguiente medida:

```
Total_Solicitudes = COUNTROWS(Solicitudes)
```

Ahora supongamos que queremos calcular el promedio de duración de resolución:

```
Promedio_Duración = AVERAGE(Solicitudes[DuraciónHoras])
```

Estas medidas pueden agregarse a gráficos, tarjetas o segmentaciones, y cambiarán dinámicamente según los filtros de comuna, mes o tipo de servicio.



Contexto de filtro

El comportamiento de una medida está determinado por el contexto: si aplicamos un filtro por año o comuna, la medida recalcula su valor dinámicamente. Por eso, las medidas son ideales para reportes comparativos, porcentajes y KPI.

Buenas prácticas



Utiliza nombres claros, con formato PascalCase o snake_case según convención local.



Documenta las medidas complejas usando comentarios DAX:

```
// Promedio de duración solo para solicitudes cerradas
Promedio_Cerradas =
CALCULATE(
    AVERAGE(Solicitudes[DuraciónHoras]),
    Solicitudes[Estado] = "Cerrada"
)
```



Agrupar medidas por categoría temática usando carpetas de visualización.



Prefiere medidas antes que columnas si el cálculo es agregado o depende del filtro.

Errores comunes

Intentar replicar lógica fila a fila con medidas, generando errores de contexto.

No entender cómo se combinan filtros desde visualizaciones, relaciones o medidas anidadas.

Hacer cálculos innecesariamente complejos en una sola expresión: es mejor usar VAR.

Las medidas calculadas son el núcleo de los análisis dinámicos en Power BI. Permiten responder preguntas del negocio con precisión, eficiencia y adaptabilidad. Dominar su lógica y sintaxis es clave para cualquier profesional que construya dashboards efectivos. En el siguiente tema aprenderemos a crear tablas completas derivadas a partir de expresiones DAX.

/* Tablas calculadas */

Creación de tablas calculadas

¿Cómo podrías generar una tabla nueva a partir de condiciones o relaciones lógicas, sin alterar los datos originales?

Las tablas calculadas son entidades creadas mediante expresiones DAX que generan una nueva tabla dentro del modelo. A diferencia de columnas o medidas, las tablas calculadas pueden contener múltiples columnas y filas, y se comportan como cualquier tabla importada desde una fuente externa.

Su principal utilidad es apoyar el modelado, crear relaciones puente, clasificaciones auxiliares o agregar tablas de fechas sin requerir una fuente externa.



Ejemplo aplicado

Supongamos que queremos una tabla que contenga únicamente las solicitudes cerradas:

```
Solicitudes_Cerradas =  
FILTER(  
    Solicitudes,  
    Solicitudes[Estado] = "Cerrada"  
)
```

O podríamos generar una tabla de resumen por tipo de servicio:

```
Resumen_Servicio =  
SUMMARIZE(  
    Solicitudes,  
    Solicitudes[TipoServicio],  
    "Total", COUNTROWS(Solicitudes)  
)
```

Estas tablas se pueden usar como origen de visualizaciones, segmentadores o para crear relaciones específicas.

Usos comunes



Tablas puente para relaciones N:N

Permiten resolver relaciones complejas entre dimensiones compartidas (por ejemplo, ciudadanos que interactúan con múltiples servicios).



Tablas resumen o de agregación

Agrupan datos en niveles superiores, optimizando la lectura y reduciendo carga de cálculo en visualizaciones.



Tablas de calendario personalizadas

Facilitan análisis por fechas fiscales, semanales, trimestres móviles, etc., cuando las fechas originales son insuficientes.



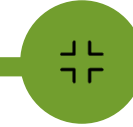
Tablas clasificatorias auxiliares

Agrupan rangos, categorías o variables calculadas que no existen en los datos fuente, como niveles de satisfacción.

Buenas prácticas



Nombrado claro y orientado a negocio: Utiliza nombres expresivos como `Puente_ServicioCiudadano` o `Tabla_Calendario_Fiscal` para mejorar el mantenimiento del modelo.



Optimiza la cardinalidad: Evita columnas innecesarias en tablas calculadas que solo agregan tamaño al modelo sin aportar valor.



Evalúa el rendimiento: Usa `CALCULATE` o `SUMMARIZECOLUMNS` en vez de `FILTER` si necesitas eficiencia en grandes volúmenes de datos.



Ocultas las tablas auxiliares si no están destinadas a ser visualizadas directamente.



Valida relaciones nuevas: Antes de conectar tablas calculadas al modelo, verifica que mantengan la unicidad y consistencia lógica esperada.

Errores frecuentes

Duplicar tablas de hechos parcialmente filtradas sin comprender las consecuencias en la lógica del modelo.

Sobreuso de tablas intermedias que podrían resolverse mediante medidas más eficientes o visualizaciones bien diseñadas.

No documentar el propósito de la tabla, dificultando su reutilización por otros equipos.

Asumir que se actualizan automáticamente: Algunas tablas calculadas pueden requerir actualización manual si se basan en columnas que cambian.

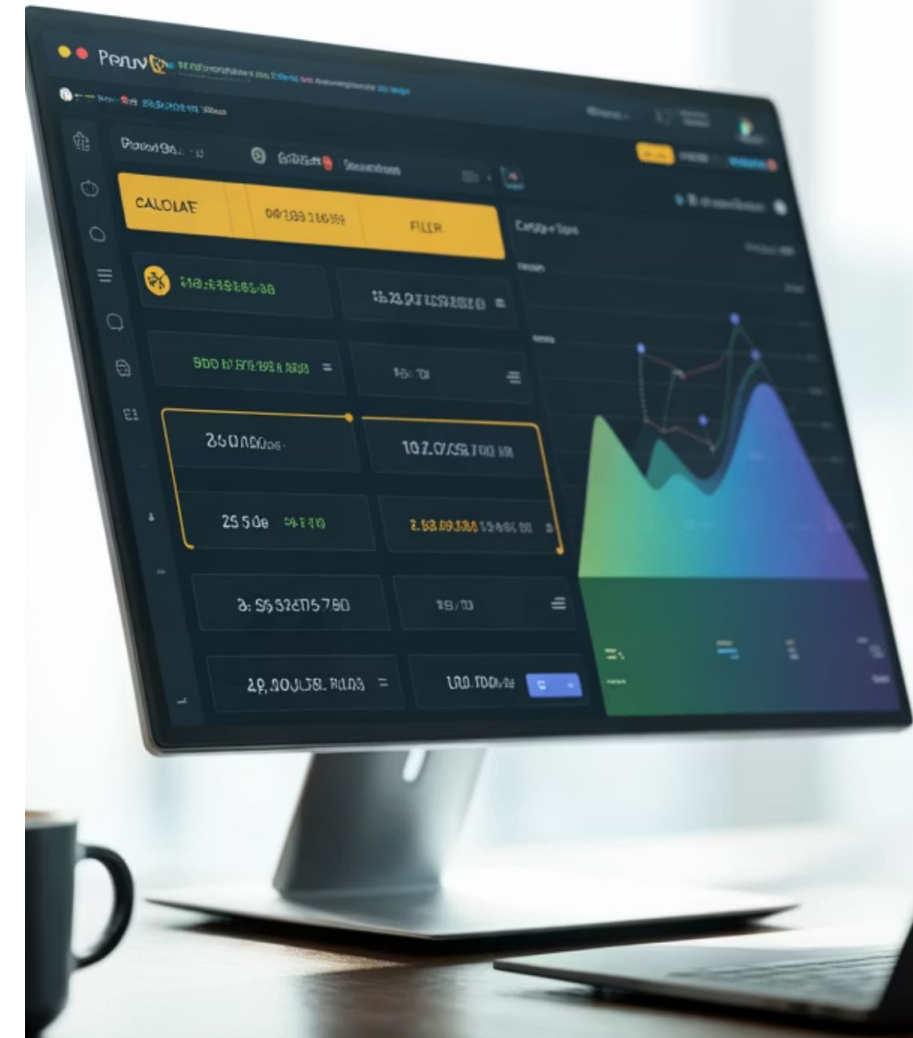
Las tablas calculadas son una herramienta poderosa para extender los modelos sin modificar las fuentes originales. Permiten implementar lógica adicional, segmentaciones específicas o tablas auxiliares. En los siguientes temas profundizaremos en las funciones más utilizadas en DAX y su aplicación práctica en contexto profesional.

/* Funciones más utilizadas */

Funciones más utilizadas (COUNTROWS, RELATED, CALCULATE, DIVIDE)

¿Cómo puedes responder a preguntas del negocio como "¿Cuántas solicitudes se han resuelto este mes?" o "¿Cuál es la proporción de solicitudes resueltas frente a las recibidas?" usando funciones DAX?

El poder de DAX reside en sus funciones. En esta sección exploramos funciones que se aplican constantemente en el análisis de datos en Power BI, cada una con una lógica específica para operar sobre columnas, relaciones, contextos y condiciones. Su dominio es esencial para realizar cálculos avanzados dentro del modelo tabular.



Funciones clave explicadas y aplicadas

1

COUNTROWS

Devuelve el número de filas de una tabla. Es útil para conteos simples y dinámicos.

```
TotalSolicitudes = COUNTROWS(Solicitudes)
```

Uso productivo: contar registros de tickets, reclamos o formularios recibidos.

2

RELATED

Permite acceder a una columna de una tabla relacionada (muchos a uno). Es útil para recuperar atributos desde una dimensión.

```
NombreComuna = RELATED(Comunas[Nombre])
```

Uso productivo: anexar información descriptiva como el nombre de la comuna desde la tabla de dimensión en una medida o columna calculada.

Funciones clave explicadas y aplicadas

1

CALCULATE

Transforma el contexto de filtro bajo el cual se evalúa una expresión. Es la función más potente y utilizada para definir medidas condicionadas.

```
SolicitudesCerradas =  
CALCULATE(  
    COUNTROWS(Solicitudes),  
    Solicitudes[Estado] = "Cerrada"  
)
```

Uso productivo: definir métricas específicas para estados, tipos de usuario o rangos de fecha.

2

DIVIDE

Divide dos expresiones y maneja errores de división por cero.

```
TasaResolución = DIVIDE([SolicitudesCerradas],  
[TotalSolicitudes])
```

Uso productivo: calcular tasas, porcentajes o razones en indicadores clave.

Buenas prácticas



Utilizar DIVIDE en lugar del operador / para evitar errores por división entre cero.



Revisar relaciones antes de usar RELATED para asegurar integridad referencial.



Encapsular condiciones dentro de CALCULATE para claridad y control del contexto de filtro.



Usar nombres descriptivos para medidas y evitar abreviaciones confusas.

Errores comunes

Intentar usar RELATED en relaciones muchos a muchos (no funciona correctamente).

Olvidar que CALCULATE cambia el contexto y puede dar resultados inesperados si no se entiende el filtro aplicado.

No validar que las medidas dependan de datos existentes: contar sin contexto o filtrar vacíos puede devolver cero inesperadamente.

Estas funciones representan la base sobre la que se construyen medidas y expresiones DAX en modelos profesionales. Su aplicación correcta permite diseñar paneles interactivos, robustos y significativos para la toma de decisiones basada en datos. En el próximo tema, abordaremos las funciones de inteligencia de tiempo, clave para comparar y analizar periodos históricos y actuales.

**/* Funciones de inteligencia de tiempo
*/**

Funciones de inteligencia de tiempo

¿Cómo puedes calcular indicadores como "acumulado mensual", "comparación con el año anterior" o "crecimiento interanual" de forma automática en Power

BI?
Las funciones de inteligencia de tiempo permiten manipular fechas y generar cálculos basados en periodos temporales. Estas funciones aprovechan una tabla de fechas (también llamada calendario) que debe estar correctamente relacionada con la tabla de hechos principal. Esto permite realizar análisis dinámicos de comportamiento temporal, como crecimiento mes a mes o comparación con periodos anteriores.

Year
2016

Year	Quarter	MonthNameShort	Total Gross	F. TOTALYTD
2016	Q1	Jan	\$27.569.558	\$27.569.558
2016	Q1	Feb		\$27.569.558
2016	Q1	Mar	\$341.268.248	\$368.837.806
2016	Q2	Apr	\$371.896.831	\$740.734.637
2016	Q2	May	\$485.126.730	\$1.225.861.367
2016	Q2	Jun	\$486.295.561	\$1.712.156.928
2016	Q3	Jul	\$55.483.770	\$1.767.640.698
2016	Q3	Aug	\$76.233.151	\$1.843.873.849
2016	Q3	Sep	\$21.420.368	\$1.865.294.217
2016	Q4	Oct		\$1.865.294.217
2016	Q4	Nov	\$478.614.952	\$2.343.909.169
2016	Q4	Dec	\$529.483.936	\$2.873.393.105
Total			\$2.873.393.105	\$2.873.393.105

Jan
Jan + Feb
Jan + Feb + Mar
Jan + Feb + Mar + Apr
...

Requisitos previos: tabla calendario

Para utilizar funciones de inteligencia de tiempo, es indispensable tener:

- Una tabla calendario con al menos una columna de fechas sin interrupciones.
- Relación activa entre la tabla de fechas y la tabla de hechos (1:N).

Ejemplo de creación de tabla calendario:

```
Calendario = CALENDAR(DATE(2020, 1, 1), DATE(2025, 12, 31))
```

Luego puedes añadir columnas como año, mes, trimestre:

```
Calendario[Año] = YEAR(Calendario[Date])
```

```
Calendario[Mes] = FORMAT(Calendario[Date], "MMMM")
```

Funciones clave y ejemplos

1

TOTALYTD (Total Year-To-Date)

Calcula acumulado anual de una medida:

```
TotalYTD_Solicitudes = TOTALYTD([TotalSolicitudes],  
Calendario[Date])
```

2

SAMEPERIODLASTYEAR

Compara el valor actual con el mismo periodo del año anterior:

```
Solicitudes_AñoAnterior = CALCULATE([TotalSolicitudes],  
SAMEPERIODLASTYEAR(Calendario[Date]))
```

3

DATEADD

Permite desplazar fechas hacia adelante o atrás:

```
Solicitudes_3MesesAntes = CALCULATE([TotalSolicitudes],  
DATEADD(Calendario[Date], -3, MONTH))
```

4

DATESMTD / DATESQTD / DATESYTD

Filtra fechas dentro del mes, trimestre o año en curso:

```
Solicitudes_MesActual = CALCULATE([TotalSolicitudes],  
DATESMTD(Calendario[Date]))
```


Aplicaciones en contextos productivos



Seguimiento mensual de solicitudes ciudadanas acumuladas.



Comparación del número de atenciones con el mismo periodo del año anterior.



Análisis de tendencias estacionales para planificación de recursos.

Buenas prácticas

- Siempre usa una tabla calendario relacionada: sin ella, las funciones devolverán resultados incorrectos.
- Establece la columna de fecha como clave única y continua (sin saltos ni duplicados).
- Utiliza marcadores de fecha (por ejemplo, IsToday, IsStartOfMonth) si se requiere control adicional.

Errores comunes

- Usar funciones de tiempo sin tabla calendario.
- No conectar la tabla calendario con los datos de hechos.
- Calcular con campos de fecha incompletos o inconsistentes.

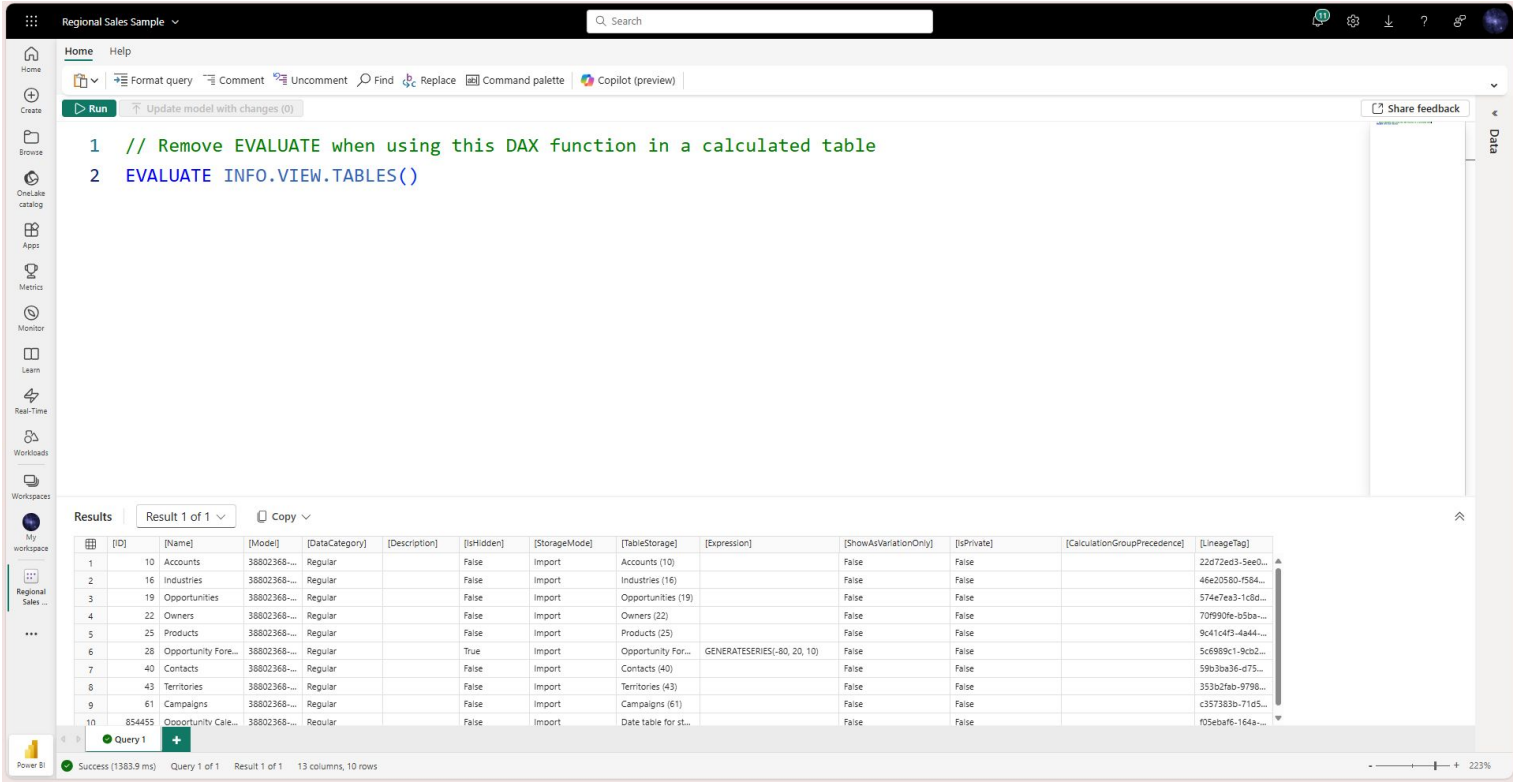
/* Funciones iterativas X */

Funciones iterativas X (SUMX, MAXX, ...)

¿Cómo puedes calcular totales, promedios u otros indicadores que dependan de la suma de valores evaluados individualmente por fila?

Las funciones iterativas de DAX permiten realizar cálculos fila por fila sobre una tabla o expresión tabular. A diferencia de funciones como SUM o AVERAGE, que operan directamente sobre columnas, las funciones iterativas permiten personalizar cómo se evalúa cada fila antes de agregarlas. Estas funciones son críticas cuando los cálculos dependen de múltiples columnas o de lógica condicional más compleja.

La mayoría de estas funciones terminan en X (por ejemplo, SUMX, AVERAGEX, MAXX, MINX, COUNTX). La X representa la capacidad de iterar.



Sintaxis general

```
FUNCIONX(Tabla, Expresión)
```

- Tabla: tabla sobre la cual se realizará la iteración.
- Expresión: operación que se evaluará en cada fila.

Funciones iterativas más comunes y ejemplos

1

SUMX

Suma valores evaluados fila por fila.

```
CostoTotal =  
SUMX(  
    Solicitudes,  
    Solicitudes[Cantidad] *  
    Solicitudes[PrecioUnitario]  
)
```

Aplicación productiva: calcular valores monetarios o métricas compuestas no disponibles directamente.

2

AVERAGEX

Calcula el promedio sobre una expresión evaluada en cada fila.

```
Promedio_Prioridad =  
AVERAGEX(Solicitudes,  
    Solicitudes[Prioridad] * 1)
```

Funciones iterativas más comunes y ejemplos

1

MAXX / MINX

Devuelven el valor máximo o mínimo evaluado fila por fila.

```
Duracion_Maxima = MAXX(Solicitudes,  
Solicitudes[DuraciónHoras])
```

2

COUNTX

Cuenta elementos evaluando condición.

```
ServiciosConSolicitudes =  
COUNTX(  
    FILTER(  
        Servicios,  
        Servicios[TotalSolicitudes] > 0  
    ),  
    Servicios[ID]  
)
```

Buenas prácticas

- Usar SUMX o AVERAGEX solo si no puedes obtener el mismo resultado con funciones agregadas simples.
- Validar la expresión que se está evaluando por fila: evita divisiones no controladas o conversiones implícitas.
- Considerar el rendimiento: estas funciones consumen más recursos, especialmente con grandes volúmenes de datos.

/* Funciones condicionales */

Funciones condicionales

¿Cómo puedes aplicar reglas de negocio que devuelvan distintos resultados según condiciones específicas del contexto?

Las funciones condicionales en DAX permiten evaluar criterios lógicos para devolver valores distintos dependiendo del resultado. Son esenciales para clasificar, etiquetar y calcular indicadores con base en condiciones particulares.

Las funciones más comunes son IF, SWITCH y IFERROR, aunque también pueden combinarse con operadores lógicos (AND, OR, NOT) y funciones como CONTAINS, ISBLANK, entre otras.

Estas funciones se utilizan ampliamente en análisis empresarial y en escenarios donde se requiere establecer umbrales o lógicas de negocio complejas.

Funcion Logica IF - Power BI Desk

InsertModelingViewHelpExternal ToolsTable toolsMeasure tools

ClasificacionHome tableRegistrosFormatText

```
1 Clasificacion =  
2 IF ( SELECTEDVALUE( Registros[Genero] ) = "Rock"; "Revisar catálogo"; "En orden" )
```

Grupo	Genero	Clasificacion
Canserbero	Rap	En orden
Deftones	Rock	Revisar catálogo
Marillion	Rock	Revisar catálogo
Miles Davis	Jazz	En orden
Whitney Houston	R&B	En orden

Funciones condicionales más utilizadas y ejemplos

1

IF

Evalúa una condición y devuelve un valor si se cumple, y otro si no.

```
Categoria_Duracion =  
IF(  
    Solicitudes[DuraciónHoras] > 48,  
    "Lenta",  
    "Rápida"  
)
```

Aplicación productiva: clasificar solicitudes según umbral de tiempo definido por la organización.

2

SWITCH

Evalúa múltiples condiciones y devuelve un resultado según el caso.

Más legible que múltiples IF anidados.

```
Clasificacion_Prioridad =  
SWITCH(  
    Solicitudes[Prioridad],  
    1, "Alta",  
    2, "Media",  
    3, "Baja",  
    "Sin definir"  
)
```


Funciones condicionales más utilizadas y ejemplos

1

IFERROR

Devuelve un valor alternativo si ocurre un error en la expresión evaluada.

```
Tasa_Resolucion =  
IFERROR(  
    DIVIDE([SolicitudesCerradas],  
    [TotalSolicitudes]),  
    0  
)
```

2

Combinación con operadores lógicos

```
Es_Urgente =  
IF(  
    Solicitudes[DuraciónHoras] < 24 &&  
    Solicitudes[Prioridad] = 1,  
    TRUE,  
    FALSE  
)
```

Buenas prácticas

- Prioriza SWITCH sobre IF cuando haya múltiples condiciones discretas.
- Asegúrate de manejar todos los casos posibles, especialmente si hay valores nulos o inesperados.
- Usa funciones condicionales dentro de medidas para construir KPIs flexibles y explicativos.
- Documenta las reglas de decisión cuando se basen en definiciones de negocio específicas.

/* Funciones de texto */

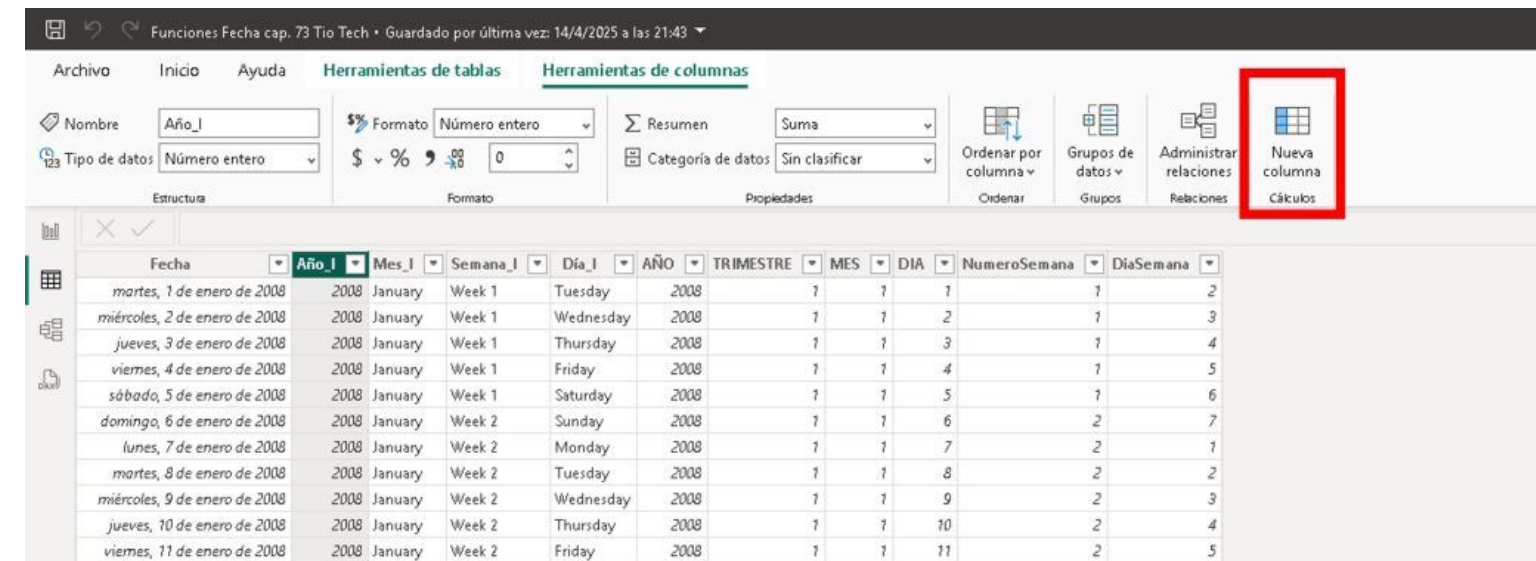
Funciones de texto

¿Cómo puedes transformar, combinar o extraer fragmentos de texto dentro de tu modelo de datos para mejorar su presentación o realizar segmentaciones más precisas?

Las funciones de texto permiten manipular y transformar cadenas de texto dentro del modelo de datos. Son útiles para crear etiquetas, clasificaciones, descripciones o estructuras de información que mejoran la legibilidad de los reportes.

Estas funciones operan sobre columnas tipo texto y permiten:

- Unir o dividir cadenas de texto
- Cambiar el formato (mayúsculas, minúsculas)
- Extraer partes específicas de una cadena
- Reemplazar valores o limpiar contenido



Funciones comunes y ejemplos aplicados

1

CONCATENATE y CONCATENATEX

CONCATENATE une dos cadenas.

```
Nombre_Completo = CONCATENATE(Solicitudes[Nombre], "  
", Solicitudes[Apellido])
```

CONCATENATEX itera sobre una tabla y concatena valores de una columna, útil en medidas.

```
ServiciosPorUsuario =  
CONCATENATEX(  
    FILTER(  
        Solicitudes,  
        Solicitudes[IDUsuario] = 123  
    ),  
    Solicitudes[Servicio],  
    ", "  
)
```

2

LEFT, RIGHT y MID

Extraen parte de una cadena:

```
Codigo_Area = LEFT(Solicitudes[Telefono], 3)
```

```
Ultimos_Digitos = RIGHT(Solicitudes[Telefono], 4)
```

```
Subcadena = MID(Solicitudes[Observaciones], 1, 20)
```

Funciones comunes y ejemplos aplicados

1

UPPER, LOWER, FORMAT

Cambian el formato de texto:

```
Nombre_Mayus = UPPER(Solicitudes[Nombre])
```

```
Nombre_Minus = LOWER(Solicitudes[Nombre])
```

```
Fecha_Formateada =  
FORMAT(Solicitudes[FechaCreacion],  
"DD-MM-YYYY")
```

2

SUBSTITUTE y REPLACE

Reemplazan caracteres o fragmentos de texto:

```
Texto_Limpio =  
SUBSTITUTE(Solicitudes[Observaciones], "-", " ")
```

Buenas prácticas

- Valida la longitud y consistencia del texto antes de aplicar funciones como LEFT, RIGHT o MID.
- Usa CONCATENATEX para evitar concatenaciones no controladas en medidas agregadas.
- Prefiere FORMAT para convertir fechas o números en texto con formatos personalizados.
- Documenta las transformaciones si implican cambios visuales importantes.

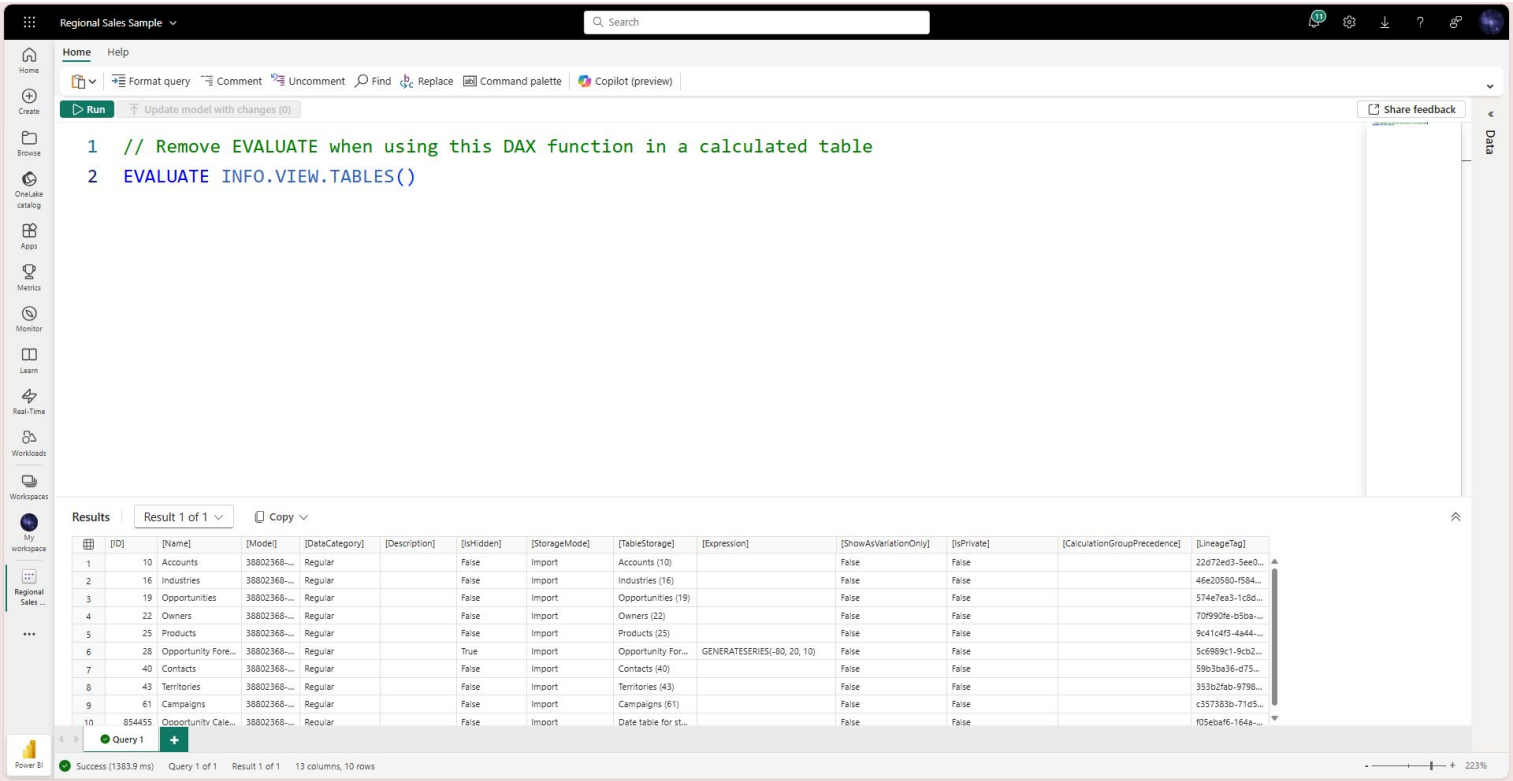
/* Funciones estadísticas */

Funciones estadísticas

¿Cómo puedes calcular indicadores como promedios, desviaciones estándar o valores máximos/mínimos para analizar el comportamiento de un conjunto de datos?

Las funciones estadísticas permiten realizar análisis descriptivos dentro del modelo de datos, ofreciendo una visión detallada del comportamiento de las variables. Estas funciones son esenciales para identificar tendencias, detectar valores atípicos, entender la dispersión y centralización de los datos, y generar alertas o clasificaciones basadas en criterios cuantitativos.

Son ampliamente utilizadas en contextos donde se requiere evaluar eficiencia, rendimiento o cumplimiento, como en servicios públicos, logística, finanzas, salud, entre otros.



Funciones estadísticas más comunes y ejemplos aplicados

1

AVERAGE

Calcula el promedio de una columna:

```
Promedio_Duracion =  
AVERAGE(Solicitudes[DuraciónHoras])
```

2

MEDIAN

Devuelve la mediana (valor central de una distribución):

```
Mediana_Duracion =  
MEDIAN(Solicitudes[DuraciónHoras])
```

3

MIN y MAX

Devuelven el valor mínimo y máximo de una columna:

```
Duracion_Minima =  
MIN(Solicitudes[DuraciónHoras])
```


Funciones estadísticas más comunes y ejemplos aplicados

1

STDEV.P y STDEV.S

Calculan la desviación estándar poblacional o muestral:

`Desviacion_Duracion = STDEV.P(Solicitudes[DuraciónHoras])`

2

VAR.P y VAR.S

Varianza poblacional y muestral:

`Varianza_Duracion = VAR.S(Solicitudes[DuraciónHoras])`

Buenas prácticas

- Verifica la calidad del dato: valores nulos, extremos o errores de registro pueden sesgar los resultados.
- Utiliza funciones estadísticas en combinación con segmentaciones o filtros para mayor precisión.
- Acompaña los valores promedio con medidas de dispersión (como desviación estándar) para una lectura más completa.
- Documenta supuestos: ¿es una muestra o una población? Esto define si usas STDEV.S o STDEV.P, por ejemplo.

Buenas prácticas con DAX

¿Qué aspectos debes considerar al momento de construir expresiones DAX complejas que deben ser eficientes, legibles y mantenibles en el tiempo?

El lenguaje DAX es poderoso, pero con gran capacidad también viene una alta responsabilidad en su uso. A medida que los modelos crecen, la calidad del código DAX impacta directamente en el rendimiento, claridad y facilidad de mantenimiento del proyecto. Aplicar buenas prácticas no solo evita errores comunes, sino que también mejora la colaboración en equipos y facilita auditorías o rediseños.

Las siguientes recomendaciones están alineadas con estándares de la industria y adaptadas a contextos productivos como inteligencia de negocios municipal, gestión de atención ciudadana, y monitoreo institucional.



Buenas prácticas estructurales

Nomenclatura descriptiva

Nombra medidas, columnas y tablas de manera expresiva y en contexto. Ejemplo: Tasa_Resolucion_Mensual en vez de Medida1.

Separación por categorías

Agrupar medidas por carpetas temáticas en el modelo (por ejemplo: "Indicadores de Gestión", "Métricas de Servicio").

Evita lógica redundante

Reutiliza medidas existentes en lugar de replicar fórmulas. Esto mejora el rendimiento y mantiene la consistencia.

Comentarios en el código

Usa doble barra (//) para anotar funciones complejas o decisiones de negocio.

```
// Calcula la tasa mensual de resolución para
servicios cerrados
Tasa_Resolucion_Mensual =
DIVIDE(
    [SolicitudesCerradas],
    [TotalSolicitudesMensuales]
)
```

Buenas prácticas de rendimiento

Minimiza columnas calculadas

Prefiere medidas, ya que las columnas se evalúan por fila y consumen memoria permanentemente.

Evita FILTER innecesarios

Usa CALCULATE o KPI prefiltrados si es posible.

Evita relaciones innecesarias

Cada relación activa agrega carga al modelo. Usa TREATAS para filtrado dinámico si es necesario.

Prefiere DIVIDE sobre /

Controla errores y mejora la estabilidad del modelo.

Buenas prácticas en diseño lógico

- Planifica el contexto de filtro: Cada medida debe evaluarse según el contexto esperado: tiempo, comuna, servicio, etc.
- Simplifica las expresiones: Divide expresiones largas en varias medidas intermedias reutilizables.
- Evita anidar demasiados IF: Usa SWITCH para lógica condicional múltiple.

```
TipoPrioridad =  
SWITCH(  
    [Prioridad],  
    1, "Alta",  
    2, "Media",  
    3, "Baja",  
    "Sin Clasificar"  
)
```

Consideraciones colaborativas

Documenta todas las medidas clave

Define propósito, autor y fecha de creación/modificación.

Comparte plantillas reutilizables

Establece estándares en el equipo para estructuras, funciones y convenciones.

Versiona los modelos

Usa nombres de versiones o respaldo antes de hacer cambios estructurales mayores.

Aplicar buenas prácticas en DAX no es opcional: es una condición necesaria para garantizar que los modelos crezcan de forma sostenible, legible y confiable. Tanto para trabajo individual como colaborativo, estas pautas mejoran la calidad del análisis y previenen errores difíciles de diagnosticar.

Ejercicio - Diseño y validación de KPIs en lenguaje DAX



Ejercicio

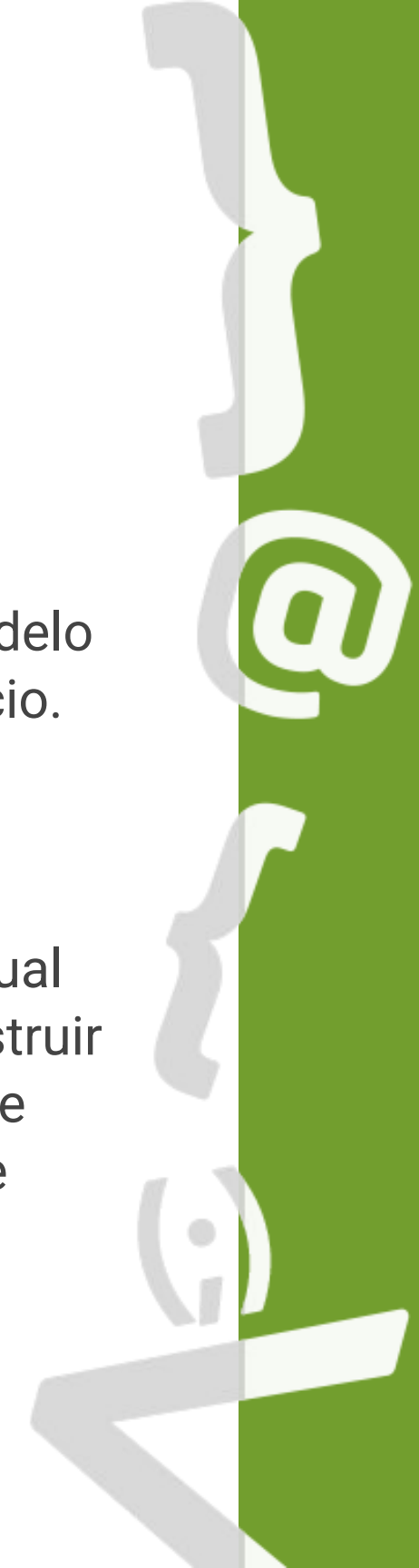
Instrucciones

Objetivo de la actividad

Aplicar funciones de DAX para construir medidas, columnas y tablas calculadas sobre un modelo de datos simulado, respetando buenas prácticas de legibilidad, rendimiento y lógica de negocio.

Situación problema

La Unidad de Atención Ciudadana de la Municipalidad desea monitorear el desempeño mensual de sus servicios a partir de datos operativos provenientes de distintas fuentes. Necesita construir un conjunto de indicadores clave de desempeño (KPIs), como tasa de resolución, promedio de atención, solicitudes por comuna y evolución mensual. Para ello, se ha solicitado al equipo de análisis crear un modelo eficiente y documentado usando lenguaje DAX.



Ejercicio

Instrucciones

Carga y modela los datos

Importa los tres archivos entregados en Power BI.

Define relaciones:

Solicitudes[FechaCreacion] con Calendario[Fecha]

Solicitudes[IDServicio] con Servicios[IDServicio]

Crea una medida para contar todas las solicitudes

```
TotalSolicitudes = COUNTROWS(Solicitudes)
```



Ejercicio

Instrucciones

Crea una medida para calcular las solicitudes cerradas

```
SolicitudesCerradas =  
CALCULATE(  
    [TotalSolicitudes],  
    Solicitudes[Estado] = "Cerrada"  
)
```

Calcula la tasa de resolución

```
TasaResolucion = DIVIDE([SolicitudesCerradas], [TotalSolicitudes])
```



Ejercicio

Instrucciones

Genera una columna para clasificar solicitudes según su duración

```
ClasificacionDuracion =  
IF(  
  Solicitudes[DuracionHoras] > 48,  
  "Lenta",  
  "Rápida"  
)
```

Crea una tabla calculada con el total de solicitudes por servicio y comuna

```
ResumenSolicitudes =  
SUMMARIZE(  
  Solicitudes,  
  Servicios[NombreServicio],  
  Solicitudes[Comuna],  
  "Total", COUNTROWS(Solicitudes)  
)
```

{desafío}
latam_



Ejercicio

Instrucciones

Agrega una medida de promedio de duración y analiza su desviación estándar

```
PromedioDuracion = AVERAGE(Solicitudes[DuracionHoras])
```

```
DesviacionDuracion = STDEV.P(Solicitudes[DuracionHoras])
```

Resumen

- Comprendimos el propósito y ventajas de utilizar el lenguaje DAX en Power BI.
- Aprendimos a crear columnas calculadas, medidas y tablas calculadas.
- Aplicamos funciones fundamentales como CALCULATE, DIVIDE, RELATED, COUNTROWS, SUMX, IF, SWITCH, FORMAT y STDEV.P.
- Utilizamos funciones de inteligencia de tiempo, funciones iterativas y funciones estadísticas.
- Implementamos buenas prácticas para construir expresiones eficientes, legibles y sostenibles en DAX.

¿Cómo puedes detectar si un cálculo DAX está generando resultados erróneos o inconsistentes?





Próxima sesión...

En la próxima sesión abordaremos la elaboración de un reporte visual. Aprenderemos a construir reportes interactivos en Power BI, incorporando distintos tipos de visualizaciones como gráficos de área, barras, KPI, embudos, medidores, mapas y esquemas jerárquicos.